1. **To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.**

   Task -1: Create Project Structure as follows:

```
OSSP/
│
├── .git/                # Git repository (created automatically)
├── .gitignore           # Files/folders Git should ignore
├── README.md            # Project description and instructions
├── LICENSE              # Project license (optional)
├── Makefile             # Build instructions
│
├── src/                 # Source code
│   ├── main.c
│   ├── shell.c
│   ├── parser.c
│   ├── executor.c
│   └── builtin.c
│
├── include/             # Header files
│   ├── shell.h
│   ├── parser.h
│   ├── executor.h
│   └── builtin.h
│
├── obj/                 # Object files (.o) (generated)
│
├── bin/                 # Executable file (generated)
│   └── my_shell
│
├── docs/                # Documentation
│   ├── design.md
│   └── report.pdf
│
├── tests/               # Test programs
│   ├── test_parser.c
│   └── test_executor.c
│
├── scripts/             # Helper scripts
│   └── run.sh
│
└── assets/              # Images/screenshots
```

└── architecture.png

Note: For creating this structure use mkdir and touch commands.

```
amrutha@blue:~$ cd ~
amrutha@blue:~$ pwd
/home/amrutha
amrutha@blue:~$ mkdir OSSP
amrutha@blue:~$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: will change to "main" in Git 3.0. To configure the initial branch name
hint: to use in all of your new repositories, which will suppress this warning,
hint: call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/amrutha/.git/
amrutha@blue:~$ mkdir src include obj bin docs tests scripts assets
amrutha@blue:~$ ls
InRelease  OSSP  assets  bin  docs  include  obj  process  process.c  pytorch-env  scripts  src  tests
amrutha@blue:~$ touch .gitignore README.md LICENSE Makefile
amrutha@blue:~$ touch src/main.c src/shell.c src/parser.c src/executor.c src/builtin.c
amrutha@blue:~$ touch include/shell.h include/parser.h include/executor.h include/builtin.h
amrutha@blue:~$ touch docs/design.md docs/report.pdf
amrutha@blue:~$ touch tests/test_parser.c tests/test_executor.c
amrutha@blue:~$ touch scripts/run.sh
amrutha@blue:~$ touch assets/architecture.png
amrutha@blue:~$ touch bin/my_shell
```

```
amrutha@blue:~/OSSP$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── fork_exec
├── fork_exec.c
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

9 directories, 20 files
```

2. **To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.**

   Task-1: Using a manual command understand the fork() and exec() system calls
   Task-2: Write a C program to create a new process and accepting the user commands using fork() and exec() system calls.

   Example:

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t pid;
    pid = fork();   // Create a new process
    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        // Child Process
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        // Parent Process
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    return 0;
}
```

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    char command[100];

    printf("Enter a Linux command: ");
    scanf("%99s", command);

    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process\n");
        printf("Child PID: %d\n", getpid());

        execlp(command, command, NULL);

        printf("Command execution failed.\n");
        exit(1);
    } else {
        printf("Parent Process\n");
        printf("Parent PID: %d\n", getpid());

        wait(NULL);

        printf("Child finished execution.\n");
    }

    return 0;
}
```

```
amrutha@blue:~/OSSP$ nano fork_exec.c
amrutha@blue:~/OSSP$ gcc fork_exec.c -o fork_exec
amrutha@blue:~/OSSP$ ./fork_exec
Enter a Linux command: pwd
Parent Process
Parent PID: 1380
Child Process
Child PID: 1385
/home/amrutha/OSSP
Child finished execution.
```